

Katholieke
Universiteit
Leuven

**Faculty of
Engineering Technology
Ghent**

GEAVANCEERDE COMPUTERARCHITECTUUR

Labo 1: Getting Acquainted with GPU Programming

Tom Thys

Contents

1 Bespreking **2**

2 Code **2**

2.1 Setup 2
2.2 CUDA setup 2
2.3 CUDA kernel 2
2.4 Uitvoeren van de CUDA-kernel 2
2.5 Programma afronden 3
2.6 Threadgrootte 3

A CUDA Source Code: flip.cu **4**

1 Bespreking

In dit labo keken we naar de basis van CUDA C++. We maakten kennis met de fundamentele principes van CUDA en gebruikten deze om een array snel en efficiënt om te draaien. Daarbij onderzochten we ook de effecten van de thread- en blockgrootte.

2 Code

2.1 Setup

Voor deze opdracht draaien we een array om. Hiervoor hebben we onze initiële array nodig die we willen omdraaien, dit doen we door een array te definiëren, met een vastgelegde grootte N, die later belangrijk zal zijn bij het bepalen van de thread- en blockgrootte.

2.2 CUDA setup

Wanneer we data willen gebruiken of schrijven op de GPU, moeten we eerst geheugen op het device reserveren. Dit doen we met:

```
cudaMalloc((void**)&d_a, N * sizeof(int));
```

waarbij `&d_a` het adres van het gereserveerde geheugen voorstelt.

Vervolgens kopiëren we onze array van de host (CPU) naar het device (GPU) met:

```
cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
```

Vanaf dit moment kan de GPU via het adres `d_a` aan onze array.

Voor de output (de omgedraaide array) reserveren we op gelijkaardige wijze geheugen op het device met enkel een `cudaMalloc`-aanroep.

2.3 CUDA kernel

Onze CUDA-kernel wordt parallel uitgevoerd: elke thread voert (bijna) gelijktijdig dezelfde code uit. We moeten er dus voor zorgen dat elke thread een unieke taak uitvoert. Hiervoor berekenen we per thread een uniek ID als volgt:

```
int idx = threadIdx.x + blockIdx.x * blockDim.x;
```

`threadIdx.x` geeft het nummer van de thread binnen een blok, `blockIdx.x` geeft het bloknummer weer, en `blockDim.x` de grootte van elk blok.

Met deze index kunnen we de array eenvoudig omdraaien:

```
b[n - idx - 1] = a[idx];
```

We voegen wel een controle toe zodat `idx < n`, om te vermijden dat we buiten de arraygrenzen lezen of schrijven. (Dit geldt uiteraard enkel als `n` effectief de grootte van de array is.)

2.4 Uitvoeren van de CUDA-kernel

We voeren onze CUDA-functie uit met:

```
flip<<<n_blocks, n_threads>>>(d_a, d_b, n);
```

waarbij we met `n_threads` het aantal threads per blok bepalen en met `n_blocks` het aantal blokken. De parameters worden als laatste meegegeven tussen de haakjes.

In onze *flip*-functie hebben we een *if*-statement toegevoegd om de waarde van `idx` te beperken. Zonder deze controle zouden er *invalid reads* en *writes* plaatsvinden zodra `n_blocks * n_threads > N`.

2.5 Programma afronden

Wanneer CUDA klaar is, kunnen we de resultaten terug naar de CPU kopiëren met:

```
cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);
```

Nu bevat array `b` de omgedraaide versie van `a`.

Tot slot moeten we het GPU-geheugen dat we eerder met `cudaMalloc` hebben gealloceerd, weer vrijgeven:

```
cudaFree(d_b);
```

2.6 Threadgrootte

Volgens de CUDA Reference Guide is het maximale aantal threads per block gelijk aan 1024. Dit limiet heeft verschillende oorzaken die te maken hebben met de architectuur van de GPU.

Een Streaming Multiprocessor (SM) moet alle threads binnen één blok gelijktijdig kunnen uitvoeren. Omdat deze threads hetzelfde *shared memory* gebruiken, moet er voldoende geheugen en rekenkracht aanwezig zijn om dit te ondersteunen.

Een blok wordt intern opgedeeld in *warps* van telkens 32 threads. De GPU voert deze warps parallel uit. Met 1024 threads per blok krijgen we dus precies 32 warps, wat goed aansluit bij de interne structuur van de SM en zorgt voor een efficiënte benutting van de hardware.

A CUDA Source Code: flip.cu

Listing 1: flip.cu

```cpp
#include <iostream>
#include <cuda_runtime.h>

// CUDA kernel flip array
__global__ void flip(int *a, int *b, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    if (idx < n) {
        b[n - idx -1] = a[idx];
    }
}

int main() {
    const int N = 10;
    int a[N], b[N];
    int *d_a, *d_b;

    // Initialize host arrays
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel (1 block of N threads)
    flip<<<1, N>>>(d_a, d_b, N);

    // Copy result back to host
    cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);

    // Print results
    std::cout << "Result:␣";
    for (int i = 0; i < N; i++) {
        std::cout << b[i] << "␣";
    }
    std::cout << std::endl;

    // Free device memory
    cudaFree(d_a);
    cudaFree(d_b);

    return 0;
}
```